

BÁO CÁO SAU BUỔI THỰC HÀNH_SESSION08

Môn học: Microservice System Design (Tiết thực hành)

Tên buổi thực hành: Mini Project - Bản ghi nhận yêu cầu khách hàng (Client Requirements Brief)

Thời gian thực hành: 120p

Họ và tên sinh viên: Lê Nhật Linh

Mã sinh viên: PTIT-HN-121

Lớp: HN-K24-CNTT2

Nhóm: 2

I. NỘI DUNG ĐÃ THỰC HÀNH

1. Phân tích bối cảnh & Đáp ứng các Nhóm yêu cầu chung của khách hàng:

- **Chuyển đổi kiến trúc (Monolithic → SOA → MSA - Session 02):**
 - Lý do chuyển đổi: Hệ thống Monolithic cũ như một "cỗ máy khổng lồ", các chức năng dùng chung codebase và database làm tăng độ phức tạp, gián đoạn toàn bộ hệ thống khi triển khai/sửa đổi. Việc tách sang Microservice Architecture (MSA) giúp các đội phát triển, kiểm thử và triển khai từng service độc lập mà không ảnh hưởng tiến độ của nhau.
- **Dữ liệu độc lập (Database-per-service - Session 02):**
 - Mỗi microservice quản lý riêng một cơ sở dữ liệu (ví dụ: patient_db, appointment_db, medical_record_db). Tuyệt đối không cho phép truy cập cross-database trực tiếp. Thay đổi schema ở một service không ảnh hưởng tới service khác.
- **Cấu hình tập trung (Centralized Configuration - Session 03):**
 - Xây dựng **Config Server** (kết nối Git/Local storage) tập trung toàn bộ thông số cấu hình (Database URL, Eureka Server URL, Gateway Routes, Server Ports, Timeouts). Đội vận hành thay đổi

cấu hình tại repository tập trung mà không phải sửa hay rebuild/re-deploy từng microservice.

- **Khả năng mở rộng & Phát hiện dịch vụ (Service Registration & Discovery - Session 03 & 05):**
 - Triển khai **Eureka Server**. Mỗi microservice khi khởi chạy sẽ tự động đăng ký thông tin (IP, Port, Service Name) với Eureka. Khi tăng/giảm bản sao (instance) hoặc di chuyển server, các service khác tự động phát hiện thông qua Eureka Client cache mà không cần chỉnh sửa cấu hình thủ công.
- **Điểm truy cập duy nhất & Cân bằng tải (API Gateway & Load Balancing - Session 05):**
 - Triển khai **Spring Cloud Gateway** làm điểm truy cập duy nhất (Single Point of Entry / Reverse Proxy) ẩn toàn bộ kiến trúc phía sau.
 - Tích hợp **Spring Cloud LoadBalancer** giúp tự động phân bổ tải (Round Robin) giữa các bản sao instance đang chạy song song.
- **Giao tiếp đồng bộ giữa các dịch vụ (Synchronous REST - Session 06):**
 - Sử dụng cơ chế gọi REST đồng bộ để gộp dữ liệu từ nhiều microservice trả về cho client trong đúng 1 lần yêu cầu.

2. Mẫu mẫu giải quyết Yêu cầu đặc biệt ("Khoai" nhất - Mục 4 Đề bài):

- **Bài toán:** Khách hàng yêu cầu: (1) Database-per-service độc lập; (2) Màn hình tổng quan phải hiển thị đầy đủ thông tin gộp từ nhiều dịch vụ trong 1 lần tải trang; (3) Mở rộng/thay đổi instance không được sửa code/config ở các service gọi; (4) Chỉ cung cấp MỘT địa chỉ truy cập duy nhất.
- **Giải pháp kỹ thuật (nằm hoàn toàn trong phạm vi Session 02 - 06):**
 - Client gửi request duy nhất tới URL của **Spring Cloud Gateway**.
 - Gateway hoặc Composite Service trung gian đóng vai trò **Data Aggregator**, sử dụng **FeignClient (OpenFeign)** kết hợp **Eureka Server** và **Spring Cloud LoadBalancer** để phát hiện vị trí các service (patient-service, appointment-service, medical-record-service) và tự động cân bằng tải.
 - Aggregator thực hiện các lệnh gọi REST đồng bộ tới các service, lấy dữ liệu độc lập từ các database riêng biệt (patient_db, appointment_db, medical_record_db), tổng hợp lại thành 1 payload JSON duy nhất và trả về cho Client.

3. Giải đáp & Bảo vệ 6 Câu hỏi gợi mở (Mục 6 Đề bài):

- [Session 02 & 05] Thành phần chịu trách nhiệm gộp dữ liệu cho màn hình tổng quan:
 - **Lựa chọn:** Sử dụng **API Gateway (Spring Cloud Gateway)** kết hợp với cơ chế Composite Service / Data Aggregator.
 - **Giải thích:** API Gateway tiếp nhận 1 request duy nhất từ phía Client, phối hợp gọi các microservices nội bộ (patient-service, appointment-service, medical-record-service), gộp dữ liệu thành 1 kết quả duy nhất trả về cho Client, đảm bảo nguyên tắc client chỉ biết 1 địa chỉ và không phải gọi nhiều lần.
- [Session 06] Thách thức khi gọi đồng bộ (Synchronous Calls) nhiều service:
 - **Thách thức:** Hiện tượng cộng dồn thời gian phản hồi (latency accumulation). Nếu 1 trong các service bị chậm hoặc sự cố, toàn bộ request tổng hợp sẽ bị hoãn hoặc timeout.
 - **Xử lý trong phạm vi đã học:** Tối ưu hóa các luồng gọi, cấu hình Read/Connect Timeout hợp lý và thực hiện xử lý ngoại lệ fallback đơn giản ở tầng ứng dụng.
- [Session 06] Lựa chọn công nghệ giữa RestTemplate và FeignClient:
 - **Lựa chọn:** FeignClient (OpenFeign).
 - **Lý do:**
 - FeignClient là Declarative REST Client (khai báo qua @FeignClient), mã nguồn ngắn gọn, dễ đọc, dễ bảo trì hơn so với RestTemplate.
 - Tích hợp sẵn với Eureka Service Discovery và Spring Cloud LoadBalancer, cho phép gọi service theo Service Name (ví dụ: PATIENT-SERVICE) mà không cần hardcode IP/Port.
- [Session 05] Hiện thực hóa "Một địa chỉ duy nhất" & Định tuyến linh hoạt:
 - **Thành phần:** Spring Cloud Gateway.
 - **Cơ chế:** Gateway làm Reverse Proxy đóng vai trò điểm truy cập duy nhất. Gateway đăng ký làm Eureka Client, tự động truy vấn bảng định tuyến các instance từ Eureka Server. Khi số lượng instance thay đổi, Gateway tự cập nhật route mà không cần can thiệp cấu hình.
- [Session 03 & 05] Phối hợp giữa Eureka Discovery và Load Balancing:

- Khi khởi chạy, mỗi instance đăng ký IP/Port với Eureka Server và duy trì heartbeat.
- Khi một service gọi service khác qua FeignClient, **Spring Cloud LoadBalancer** sẽ lấy danh sách các instance UP từ Eureka Client cache và điều phối tải (Round Robin). Do đó việc thêm/bớt instance diễn ra hoàn toàn minh bạch, không cần sửa bất kỳ dòng config nào của service gọi.
- **[Session 03] Quản lý cấu hình tập trung với Config Server:**
 - **Các thông số tập trung:** Datasource URL, Database Credentials, Eureka Registry URL, Gateway Routing Rules, Server Ports, Read/Connect Timeouts.
 - **Lợi ích:** Tách biệt hoàn toàn Code và Configuration (12-Factor App). Giúp đổi môi trường hoặc tham số vận hành tức thì mà không cần rebuild/re-deploy service.

4. Liệt kê câu hỏi & phản biện từ nhóm khác:

5. Thực hành triển khai code:

- Khởi tạo & Cấu hình **Config Server** kết hợp file cấu hình tập trung.
- Triển khai **Eureka Server** làm Service Discovery.
- Xây dựng các microservices (patient-service, appointment-service, medical-record-service) tự đăng ký với Eureka.
- Cấu hình **Spring Cloud Gateway** và **Spring Cloud LoadBalancer**.

Dùng **FeignClient** thực hiện giao tiếp đồng bộ, gộp dữ liệu thành công cho màn hình tổng quan MediCore qua Postman/Gateway.

II. CÔNG VIỆC CÁC EM ĐÃ LÀM

1. Công việc cá nhân:

- Phân tích yêu cầu bài toán khách hàng từ Client Requirements Brief.
- Trả lời chi tiết và bảo vệ 6 câu hỏi gợi mở theo phạm vi kiến thức Session 02 - Session 06.
- Xây dựng FeignClient và cấu hình Gateway, Config Server.

Hoàn thiện báo cáo thực hành Session 08.

2. Công việc nhóm:

- Thảo luận kiến trúc, phân chia các microservices và ranh giới dữ liệu (Database-per-service).

Thực nghiệm nhân bản instance, kiểm thử Cân bằng tải (Load Balancing) và Gateway Routing.

III. KẾT QUẢ CÁC EM ĐẠT ĐƯỢC

- **Link mini-project GitHub:**
https://github.com/LINH-2006-CUTE/PTIT_CNTT2-K24_IT214_Session08

Triển khai thành công hệ thống Microservices đáp ứng đầy đủ yêu cầu của khách hàng trong phạm vi kiến thức đã học.

IV. KHÓ KHĂN VÀ VẤN ĐỀ CÁC EM GẶP PHẢI

- Việc duy trì nguyên tắc Database-per-service đòi hỏi thiết kế dữ liệu chặt chẽ.
- Xử lý bài toán gộp dữ liệu (Data Aggregation) từ nhiều FeignClient response trong điều kiện gọi đồng bộ mà không vi phạm quy tắc giới hạn công nghệ của bài học.

Cấu hình khớp nối chính xác giữa Config Server, Eureka Registry và Gateway Route Locator.

V. KINH NGHIỆM RÚT RA

1. Kinh nghiệm rút ra từ bài thực hành:

- Hiểu rõ bản chất chuyển đổi từ Monolithic sang Microservices và cách áp dụng tư duy hệ thống phân tán.
- Nắm vững sự phối hợp giữa Config Server, Eureka Server, API Gateway, Spring Cloud LoadBalancer và FeignClient.
- Dùng tư duy kỹ thuật để chuyển đổi yêu cầu nghiệp vụ thực tế của khách hàng thành mô hình kiến trúc chuẩn xác.

2. Lưu ý cho những buổi thực hành sau:

- Chú ý cấu hình Timeout cho FeignClient để tránh nghẽn luồng.

Kiểm tra trạng thái UP trên Eureka Dashboard trước khi kiểm thử routing qua Gateway.

VI. ĐỀ XUẤT / KIẾN NGHỊ

Không

VII. KẾT LUẬN

1. Áp dụng: Tốt

2. Khả năng hiểu bài: Ổn